

A Comparative Study of Task Offloading Approaches in the Edge-Cloud Paradigm

Gurman Kaur

March 29, 2025

Contents

1	Introduction	3
1.1	Scope and Motivation	3
1.2	Problem Definition	3
1.3	Project Objectives	4
1.4	Project Contributions	4
1.5	Project Organization	5
2	Background and Related Work	6
2.1	Edge of Things: The Big Picture on the Integration of Edge, IoT and the Cloud in a Distributed Computing Environment	6
2.2	All one needs to know about fog computing and related edge computing paradigms: A complete survey	8
2.3	A Survey on Edge and Edge-Cloud Computing Assisted Cyber-Physical Systems . . .	10
2.4	Fuzzy Workload Orchestration for Edge Computing	11
2.5	A novel approach for IoT tasks offloading in edge-cloud environments	13
2.6	Adaptive Code Offloading for Mobile Cloud Applications: Exploiting Fuzzy Sets and Evidence-based Learning	15
3	Analytical Study of the Task Offloading Algorithms	19
3.1	Description of the Task offloading approaches	19
3.1.1	Next Fit	19
3.1.2	Network based, Utilization based, Hybrid	19
3.1.3	Fuzzy	20
4	Analysis of the Task offloading approaches	21
4.0.1	Number of Failed tasks	21
4.0.2	Network Delay	22
4.0.3	Processing Delay	23
4.0.4	Service time	23
4.0.5	Edge Server Utilization	24
4.0.6	Cloud Server Utilization	25
5	Conclusion	26

1 Introduction

Cloud computing has transformed the way data is processed, stored, and managed by providing scalable, on-demand computational resources. However, the cloud's centralized nature often results in high network latency, increased bandwidth consumption, and potential service disruptions, making it unsuitable for real-time applications such as autonomous driving, smart cities, and industrial automation.

Edge computing tackles these challenges by bringing computation and data processing closer to the data source. By processing data at the network's edge- on IoT devices, local gateways, and edge servers edge computing significantly reduces latency, optimizes resource utilization, and improves quality of service (QoS) for applications like smart grids and real-time analytics.

The edge computing does not entirely replace cloud computing but rather integrates with it to form edge-cloud-computing. The approach combines real-time processing of edge with the cloud's large-scale storage and computing power. However, efficient task scheduling and resource management remain key challenges. This paper examines various existing task-offloading strategies and provides a detailed analysis of each approach.

1.1 Scope and Motivation

This study focuses on analyzing existing task-offloading strategies in cloud-edge systems. It reviews five algorithms based on various performance metrics to investigate the most effective approach for efficient task allocation. It focuses on balancing computational loads between edge and cloud layers, minimizing network congestion, and enhancing the Quality of Service (QoS) edge environments with limited resources.

Traditional centralized cloud models have serious issues like high latency, heavy data traffic, and inefficiency in handling real-time tasks. Edge computing mitigates these issues by processing data at the network edge, reducing response time and reliance on central servers. For example, edge systems can enable real-time medical decision-making in healthcare and instant response to traffic in transportation. This study is motivated by the need to overcome these challenges and leverage the advantages of edge computing by analyzing the most efficient task-offloading mechanism that enhances computational speed and minimizes network delays.

1.2 Problem Definition

keep it as it is

Organizations are increasingly drawn to the potential benefits of scalability, resilience, and cost reduction as cloud technology adoption continues to rise. The dynamic and intricate nature of cloud environments, on the other hand, presents challenges in ensuring both cost-effectiveness and consistent resource availability. While many researchers are focused on optimizing resource utilization and lowering costs using techniques such as virtual machine allocation, the interplay between cost and availability in a multi-cloud environment remains a complex and unexplored territory.

This research project focuses on the creation of an optimization model, CAO VIA (Cost and Availability based Optimization for Virtual Machine Allocation), to improve virtual machine allocation across various Cloud Service Providers (CSPs). The primary goals include minimizing client costs while maintaining the required level of performance while taking into account the availability factors associated with various CSPs.

The difficulty comes from navigating the complexities of multi-cloud environments, where the cost of deploying virtual machines is influenced by factors like CSP pricing models, resource availability, and performance metrics. Furthermore, striking a delicate balance between cost reduction and maintaining

performance levels necessitates a nuanced understanding of CSP offerings and the changing demands of cloud users.

Addressing this challenge entails developing an optimization strategy that takes both cost and availability into account. The study will look at how workloads, pricing structures, and service level agreements differ between CSPs and how they affect the cost-effectiveness and availability of virtual machine allocations. The ultimate goal is to provide users with efficient, value-driven resource allocation, taking into account the dynamic and diverse landscape of multi-cloud environments.

This problem definition establishes the groundwork for investigating the complex relationship between cost and availability in the context of virtual machine allocation across multiple cloud providers, emphasizing the need for a comprehensive optimization model to navigate this treacherous terrain.

1.3 Project Objectives

The project focuses on analyzing existing literature on task offloading mechanisms within edge cloud environments and identifying challenges such as resource management, network dynamics, and scalability. It offers an in-depth analysis of various existing task-offloading algorithms, evaluating their performance under different conditions. The study aims to identify the most efficient offloading strategy to optimize task allocation, minimize network delays, reduce task failure rates, and enhance overall service efficiency.

1.4 Project Contributions

keep it as it is

The significant contributions of this project stem from its commitment to optimising cloud resource allocation, enhancing the user-centric model, and integrating cost and availability:

- **Establishment of a Communication Framework:** Created a strong communication framework that ensures secure and efficient user-to-CSP interaction. Impact: Improved overall user experience, laying the groundwork for more seamless data exchange and more efficient cloud resource allocation.
- **Cost Function Integration:** Created an innovative project-specific cost function for systematically evaluating CSP costs. Impact: Increased decision-making accuracy through the implementation of a structured approach to processing and integrating CSP cost data for more informed resource allocations.
- **Real-Time Availability Data Integration:** Added real-time availability data from the CLOUDSIM simulator to the optimization algorithm. Impact: Added dynamic decision-making capabilities, allowing for real-time adjustments based on the most recent availability metrics, resulting in more reliable and adaptive resource allocations.
- **Algorithm for Advanced Resource Allocation:** Created a sophisticated optimisation algorithm that takes into account both cost and availability data. Impact: Personalised and efficient resource allocation recommendations were provided to users, optimising cloud resource utilisation based on changing user needs.

These collective contributions make a significant contribution towards the goal of optimising cloud resource allocation. This project has a long-term impact by seamlessly integrating cost functions and availability data, addressing the changing needs of users in a dynamic cloud environment and contributing to the efficiency and tailoring of cloud resource allocation solutions.

1.5 Project Organization

The project is organized as follows: Chapter 2 covers the edge cloud architecture, background, and related work. Chapter 3 explains various task-offloading algorithms and an in-depth performance analysis. Chapter 4

2 Background and Related Work

//This section provides an in-depth analysis of the edge-cloud paradigm’s architecture, particularly emphasizing task filtering and offloading mechanisms within edge-computing environments.

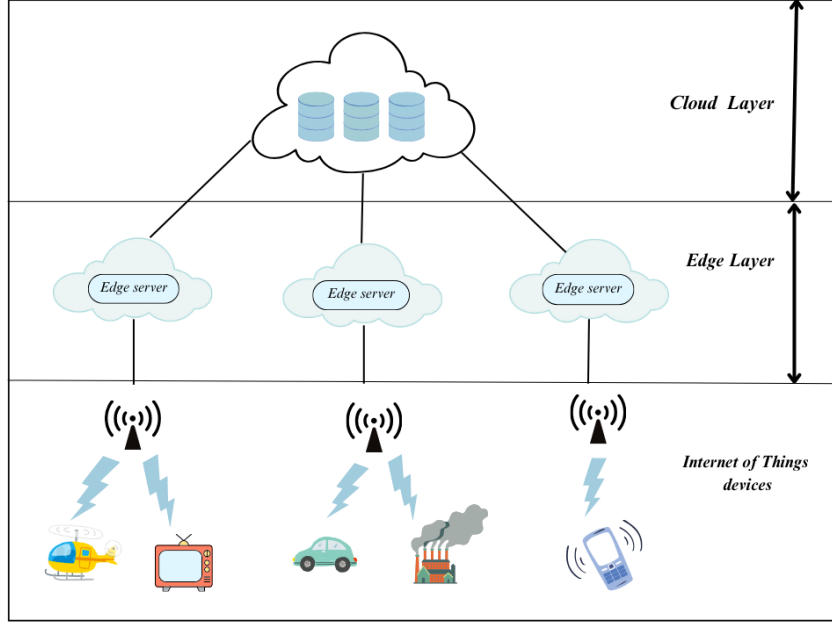


Figure 1: Edge-Cloud Architecture

The Edge-Cloud Computing Architecture is illustrated in Figure 1, where the task-offloading problem is emphasized. IoT devices such as security cameras, drones, and vehicles generate tasks, which are offloaded to edge servers, ensuring low latency. Edge servers, which are located close to the edge servers, act as a bridge between terminal devices and centralized cloud servers, providing local computation, networking, and storage services. When the task is offloaded, the edge server decides whether to execute the task entirely locally, on a remote edge server, or offload to a cloud server for execution. The decision is influenced by the task requirements, the available computational resources at the edge and cloud servers, and network conditions. Upon execution, the result is sent back to the IoT device, directly from the edge server or after cloud server assistance. This architecture reduces the dependency on cloud servers, minimizes latency, and brings computation closer to the data source to facilitate efficient real-time processing.

Managing resources efficiently is crucial, especially during high workloads. The goal is to optimally utilize computational capacity effectively across devices, edge servers, and cloud servers without overloading any component or causing delays. A dynamic approach ensures smooth task processing, low latency, and a better user experience in edge-cloud systems.

2.1 Edge of Things: The Big Picture on the Integration of Edge, IoT and the Cloud in a Distributed Computing Environment

The survey paper presents Edge Computing(EC) as a transformative approach to enhance mission-critical applications through real-time data processing, reduced latency, and high scalability. Unlike traditional cloud computing, EC processes data closer to its source- on-edge devices like smartphones,

sensors, and IoT gadgets leading to improved network response times and resource utilization. With significant growth in the adoption of the Internet of Things(IoT), EC is becoming vital in industries such as healthcare, transportation, and social networking, helping optimize services and reduce bandwidth demands. This paper examines key challenges in Edge Computing (EC), such as efficient resource management and its seamless integration with existing cloud systems, providing insights into EC's comparative performance and future research needs to enhance its scalability, efficiency, and interoperability.

Edge Computing brings data processing closer to IoT devices by shifting it from centralized cloud servers to the network's edge, reducing latency and bandwidth limitations. This decentralized approach is ideal for IoT applications that require real-time processing and decision-making. However, EC is still evolving and must overcome several network-specific challenges to optimize performance. The paper highlights key issues such as device selection, computation offloading management, and the development of reliable protocols for distributed networks like VANETs and MANETs. Each EC application requires a tailored solution to support the network demands. [1].

The survey highlights key challenges in implementing Edge Computing(EC) across diverse network environments. Device Selection is a crucial factor, especially in VANETs, where using vehicles as edge nodes ensures distributed computation but raises implementation costs and processing demands. Computation Offloading is another critical challenge, as workloads must be dynamically distributed to prevent overloading specific nodes, which can lead to performance degradation and battery drain. Task Scheduling between edge and cloud remains complex since EC cannot entirely replace cloud computing; some computations still require centralized processing for reliability and data storage. Network Standardization is essential to minimize communication overhead and prevent QoS issues like network congestion and denial of service. Additionally, Mobility Management is crucial for maintaining connectivity in high-mobility networks like MANETs and VANETs, where frequent disconnections can disrupt real-time decision-making and data processing. Security and Privacy remain important concerns, as data processed at the edge is more vulnerable to attacks.

The survey identifies key challenges associated with implementing EC in various network environments. Resource Management is a major concern, especially in networks like VANETs, where edge devices (vehicles) increase costs and processing demands. Computation Offloading presents another challenge, as workloads need to be dynamically distributed across edge devices to avoid overloading certain nodes. Task Scheduling between the cloud and edge is also complex; while EC handles real-time processing, some tasks still rely on the cloud for data storage and reliability. Additionally, Network Standardization is necessary to prevent traffic congestion and QoS issues, and Mobility Management is crucial for maintaining connectivity in high-mobility networks like VANETs and MANETs. Security and Privacy remain important concerns, as data processed at the edge is more vulnerable to attacks [1].

The paper presents a comparative analysis of Edge computing and related paradigms like Fog Computing (FC) and Cloud Computing (CC). While both EC and FC architecture push the intelligence closer to the data source, FC processes data within the local area network(LAN) and EC enables real-time analysis by connecting directly to end-user devices. CC, in contrast, operates as a centralized model with greater computational power but struggles with latency-sensitive applications. Multi-Cloud Computing(MCC) expands CC's capabilities by distributing workloads across multiple cloud servers, enhancing redundancy and fault tolerance. Compared to cloud models, EC excels in low-latency environments, making it ideal for real-time IoT applications where quick decision-making is crucial.

The role of EC is emphasized in latency-sensitive domains such as augmented reality(AR), smart homes, and vehicular networks(VANETs). Augmented Reality (AR) benefits from EC's real-time processing, while smart home systems enhance responsiveness and privacy. In smart cities, EC aids disaster response by swiftly detecting and addressing emergencies. Vehicular Networks (VANETs)

utilize EC for scheduling and load balancing, managing complex tasks efficiently. In healthcare, EC enables location-based emergency responses for quicker and more effective care. The study underscores EC's versatility, positioning it as a powerful, low-latency computing solution across industries.

A key challenge in EC is energy efficiency, particularly for IoT devices with limited power. EC reduces energy consumption by filtering and processing data locally, minimizing the need for constant cloud communication and preserving battery life. The paper discusses load balancing and adaptive task scheduling as essential techniques to optimize power usage, particularly in VANETs and smart home applications. Efficient resource allocation and selective computation offloading further improve EC's ability to manage workloads while conserving energy.

Despite its benefits, EC introduces security and privacy challenges due to its decentralized nature. Data processed at the network's edge is vulnerable to cyber threats, especially if data is processed locally rather than in a centralized, secure cloud environment. Existing encryption mechanisms are often inadequate, necessitating lightweight authentication protocols and trust management systems to secure communication between EC devices. The paper proposes using lightweight authentication mechanisms and trust management systems to enhance the security of communication between EC devices and safeguard user data. Given the dynamic nature of VANETs and MANETs, EC must incorporate real-time authentication strategies to ensure seamless and secure interactions across different edge servers. Additionally, the paper emphasizes the importance of strong data protection measures, as smart applications continuously generate large volumes of sensitive and personalized information that require robust security protocols. [1].

The paper concludes that Edge Computing(EC) is a crucial solution for real-time and latency-sensitivity IoT applications, outperforming traditional cloud models through decentralized and localized processing. While EC, Fog Computing(FC), and Cloud Computing(CC) each have distinct advantages, EC emerges as the most suitable for IoT scenarios requiring rapid response and efficient resource management. Future IoT deployments must focus on selecting the right computing technology, addressing EC's high implementation costs, and ensuring security will be essential to fully realizing the potential of distributed edge-based architectures.

2.2 All one needs to know about fog computing and related edge computing paradigms: A complete survey

The survey paper explores the challenges of processing massive amounts of data generated by IoT devices, highlighting the limitations of cloud computing in handling large volumes of data generated by time-sensitive applications such as patient monitoring or self-driving cars, which require low-latency and location-aware services, emphasizing the need for local processing. It discusses fog computing as a solution, highlighting its ability to enable computing, storage, and data management closer to IoT devices, addressing bandwidth, latency, and privacy concerns. The paper compares fog computing with related paradigms such as edge computing, mist computing, cloud of things, and cloudlets, emphasizing fog computing's flexibility and comprehensive scope.

The paper outlines a number of computing paradigms that have evolved to address various application requirements. Cloud Computing provides on-demand access to shared computing resources, offers services such as IaaS, PaaS, and SaaS and is deployed via private, community, public, or hybrid clouds. However, as the volume of data at the network edge increases, computing paradigms closer to the data sources are needed to support high-bandwidth, low-latency, and privacy-sensitive applications. Mobile Computing, enables data processing on portable devices like smartphones and laptops, supports context-aware applications and operates without a centralized hub. Mobile cloud computing (MCC) integrates cloud services with mobile computing, shifting data storage and processing to the cloud to enhance battery life and enable high computation applications like augmented reality. However, MCC faces challenges like reliance on constant internet connectivity and high latency

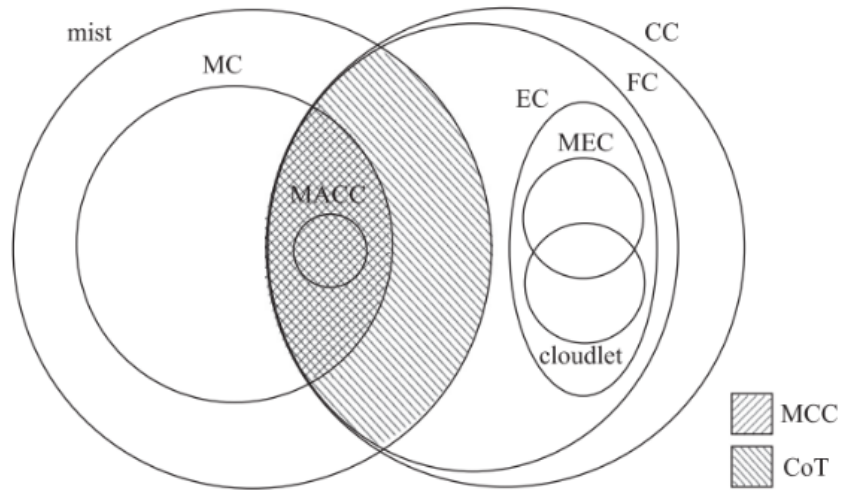


Figure 2: A classification of scope of fog computing and its related computing paradigms. (Intersection of cloud computing and mist computing is Cloud of Things, and intersection of mobile computing and cloud computing is mobile cloud computing.)[2]

for delay-sensitive applications. Mobile ad hoc cloud computing (MACC) enables decentralized computing, suitable for dynamic topologies without guaranteed internet access. Edge computing brings data processing closer to the IoT devices; at the network's edge which helps with latency, privacy, and connectivity issues. Multi-access edge computing (MEC) extends mobile computing by integrating cloud capabilities into the Radio Access Network (RAN), supporting low-latency applications, and using technologies like 5G, SDN, and NFV for better resource management. cloudlet computing reduces latency and energy consumption by providing small data centers near mobile devices. It forms a three-tier structure (mobile device–cloudlet–cloud) and supports real-time processing by offloading tasks to nearby cloudlets. Mist computing operates at the extreme network edge, enabling data processing directly on IoT devices, improving privacy, reducing latency, and minimizing dependence on the cloud. Cloud of Things (CoT) creates a virtualized cloud infrastructure using IoT devices, pooling their resources for distributed computing. CoT enhances cloud capabilities, supports real-time data processing, and is more scalable than mist computing, which focuses on individual devices. The edge cloud further extends cloud capabilities at the edge by leveraging user or operator-contributed compute nodes at the edge of the network.

Fog computing is a horizontal, system-level architecture that bridges the gap between centralized cloud services and IoT devices by distributing computation, storage, control, and networking closer to users.[5] It employs various fog nodes such as small servers, routers, switches, gateways, and set-top boxes that are deployed near IoT nodes to process data locally, thus reducing latency and bandwidth usage. For example, in Intelligent Transportation Systems, GPS data can be compressed at the edge before being sent to the cloud. The architecture also supports a fog-cloud federation where fog nodes handle real-time data filtering, preprocessing, and aggregation, while the cloud manages heavy analytical tasks and centralized decision-making. An orchestrator, often Software Defined Networking (SDN), ensures seamless cooperation between the two layers. Additionally, Fog Radio Access Networks (F-RAN) integrate fog computing into mobile networks, providing localized caching at the radio access level, significantly enhancing content retrieval speeds, and reducing the load on front-haul networks. In contrast, C-Cloud-RAN (C-RAN) centralizes control of F-RAN nodes by deploying and connecting low-cost remote radio heads (RRHs) to a centralized baseband unit (BBU) pool. The fog computing architecture is versatile and scalable, capable of operating independently with limited or intermittent

connectivity to the cloud. Its horizontal nature facilitates inter-platform interactions, allowing a mix of heterogeneous devices to collaborate in preprocessing data close to the source. Also, fog computing improves performance for mobile users by supporting broader traffic distribution along the thing-to-cloud continuum. This makes fog computing an ideal solution for latency-sensitive, high-bandwidth, and privacy-critical applications across various industries such as transportation, healthcare, and smart cities.

This study emphasizes the strengths and weaknesses of different cloud computing paradigms, focusing on increasing demand for localized computing solutions to address IoT challenges, real-time data, and latency-sensitive applications. Fog computing is presented as a versatile, well-suited solution for IoT and low-latency needs. While paradigms like MACC, mist, and CoT are more suitable for specific cases, fog computing provides a broader, more general solution across the thing-to-cloud continuum. Its integration with open standards, like the IEEE OpenFog Reference Architecture, is shaping the future of connected computing.[2]

2.3 A Survey on Edge and Edge-Cloud Computing Assisted Cyber-Physical Systems

The rapid advancement of information technologies has led to the emergence of cyber-physical systems(CPS), integrating physical objects with software components to create systems that can interact with the physical world while enabling data-driven decision-making. The system enhances various domains such as smart grids, healthcare, and manufacturing. However, the increasing data volume challenges CPS deployment, as cloud computing alone struggles with high latency and limited bandwidth. To address these limitations, edge and edge cloud computing have emerged which moved computation closer to data sources, promised improvements in CPS quality of service (QoS) by reducing latency, managed energy consumption, and enhanced security, privacy, and reliability. The research paper surveys the latest developments in edge and edge-cloud computing within CPS, addressing challenges in latency, security, and resource management to achieve an optimal QoS while reviewing state-of-the-art optimization techniques.

Edge computing and edge-cloud computing represent advanced distributed architectures that bring processing and storage closer to data sources, reducing reliance on distant cloud servers for handling large-scale data transfers. Different from traditional centralized cloud models, edge computing deploys local servers near base stations to manage networking, computation, and storage, significantly minimizing latency and easing bandwidth constraints. On the other hand, edge-cloud computing integrates both edge and cloud resources, allowing local execution of complex tasks while deferring global decision-making tasks to powerful, centralized cloud servers. The integration of emerging technologies, such as cloud-native applications, further enhances edge- cloud computing by enabling containerized, microservice-based architectures that operate seamlessly across both environments. This enhances scalability and interoperability, making it particularly beneficial for cyber-physical systems(CPS). The paper also explores edge-cloud-assisted CPS, which by combining physical and virtual elements, supports real-time applications like intelligent transportation, vehicular systems, and disease detection. It ensures efficient, distributed data collection and processing, optimizing performance and responsiveness for latency-sensitive tasks[3].

The integration of CPS with edge and edge-cloud computing introduces several challenges. First, while edge computing reduces latency by processing data closer to the source, managing large-scale CPS with a large number of connected devices requires efficient resource allocation, dynamic scheduling, and architectural designs, which complicates resource allocation and scalability. Energy consumption is another concern; unlike cloud computing, edge computing requires devices to perform preprocessing, increasing energy consumption and complicating energy management. Security and privacy risks are also important, as task offloading introduces risks related to secure data transmission, device authentication, and data privacy in open networks. Additionally, system reliability is

important for CPS, as failures in edge or cloud servers can disrupt critical applications like intelligent transportation, which requires high system reliability. [3].

To mitigate latency issues in CPS integrated with edge and edge-cloud computing, researchers have come up with various optimization techniques. In edge-assisted CPS, methods like base station alliances, resource clustering, and machine learning speed up data analysis and task processing on edge servers. For latency-sensitive domains such as healthcare and transportation, lightweight models and optimized architectures enhance low-latency performance. Meanwhile, edge-cloud computing-assisted CPS employs distributed architectures using a combination of both edge and cloud resources, delegating immediate processing to edge servers while using cloud resources for complex computations, ensuring a balanced and efficient workflow. Energy efficiency is another critical area of research, particularly for battery operated or industrial CPS applications. In edge computing, strategies like threshold-based optimization, probabilistic modeling, and selective offloading reduce energy consumption by selectively processing or deferring tasks, and also by considering the unique requirements of battery-powered devices. In edge-cloud systems, integrating renewable energy and selective offloading helps balance power usage while maintaining quality of service(QoS). These approaches extend battery life for mobile CPS devices and minimize operational costs for industrial-scale deployments. To address the dual challenge of latency and energy consumption for CPS applications, multi-objective optimization techniques such as genetic algorithm and problem decomposition are used. These methods enable energy-efficient task scheduling by adapting to system conditions. Additionally, edge computing can distribute tasks across nearby devices, optimizing energy usage while minimizing delays, particularly in interactive applications. Security and privacy remain the major concerns in distributed edge cloud CPS. Researchers have implemented techniques like data perturbation, blockchain, and contract theory to secure communications and transactions. Data perturbation adds security layers to data, while blockchain strengthens trust and integrity in applications like vehicle-to-grid energy trading. Secure cellular nodes further ensure reliable access and identity management for sensitive applications like healthcare and intelligent transportation. However, these security strategies must account for the real-time demands and energy consumption of CPS applications. System reliability is another key focus, with solutions emphasizing fault tolerance and redundancy. Edge computing architecture incorporates redundancy through hardware replication and state preservation to provide automatic recovery from transient without service interruptions. With edge-cloud environments, reliability allocation, and dynamic monitoring help ensure stable system performance across diverse scenarios, including industrial automation and disaster response. Techniques such as Monte Carlo simulations and application state preservation further enables operational continuity despite failures.[3].

The survey highlights insights from current research, noting that while edge computing effectively reduces latency, it consumes more energy than traditional cloud computing, posing scalability challenges. It offers a comprehensive review of CPS integration with edge and edge-cloud computing, covering solutions for key challenges in latency, energy efficiency, security, privacy, and reliability. The paper also highlights current capabilities and limitations in CPS design and QoS optimization. The authors suggest that future research should focus on human-behavior-aware solutions, next-generation communication (5G and beyond) communication, and quantum-resistant security strategies, to enhance efficiency, resilience, and security in edge-cloud-assisted CPS applications.

2.4 Fuzzy Workload Orchestration for Edge Computing

The rise of mobile devices, self-driving cars, augmented reality, instant video processing, tactile Internet, e, health, and Industry 4.0 has led to a significant growth of data-intensive applications requiring real-time or semi-real-time processing and involving high user mobility. Traditional cloud computing experiences network congestion and network delays, which makes it unsuitable for these highly dynamic and mobility-driven applications. Edge computing addresses this challenge by bringing com-

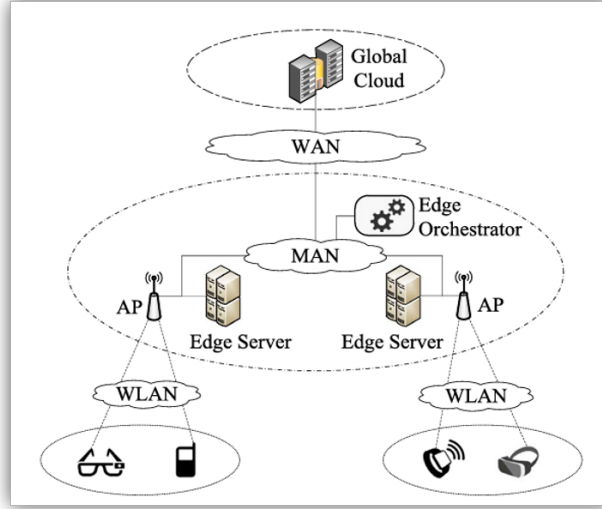


Figure 3: Multi-tier Edge Computing Architecture with Edge Orchestrator [4]

putation closer to data sources, reducing delays, and easing network congestion. However, efficiently managing workloads in a heterogeneous computing environment is complex due to fluctuating network conditions, dynamic workloads, and diverse application requirements. Determining the best place for the task execution or workload orchestration problem continues to be a major obstacle to ensuring scalability, resource efficiency, and service quality. To address this challenge, this research proposes a fuzzy logic-based workload orchestration system that mimics human decision-making, allowing adaptive and real-time task allocation based on network conditions and computational capacity for optimal performance. The system is evaluated using EdgeCloudSim, a simulation tool to demonstrate that this approach outperforms traditional methods in load balancing, reducing delays, and optimizing resource utilization.

The system model in Figure 3 comprises a multi-tier architecture designed to efficiently handle tasks generated by mobile devices. The mobile devices positioned at the first tier use wireless transmission interference to interact with wireless local area network (WLAN) gateways. The edge servers and edge orchestrators at the second tier are connected via a metropolitan area network (MAN); the edge orchestrator manages the status of edge servers and monitors the network resources used for workload orchestration. At the third tier, global cloud servers are connected through wide area networks (WAN), enabling the offloading of tasks that cannot be processed efficiently at the edge.

The orchestration between these three tiers- mobile devices, edge servers, and cloud servers ensures optimal resource utilization and task processing. This setup allows tasks to be handled either at the nearby server or a remote server or sent to the cloud, depending on the available network and server capabilities. [4].

The proposed solution for task offloading is utilizing a Fuzzy Logic System workload orchestrator as shown in Fig.4, to address the uncertainty and dynamic nature of edge computing environments. The workload orchestrator operates in two stages: first, it selects the most suitable edge server based on MAN delay, server utilization, and task characteristics; second, it determines whether the task should be processed locally at the edge or offloaded to the cloud. The Fuzzy Inference System (FIS) consists of three key components: a singleton fuzzifier- which maps multiple inputs such as WAN bandwidth, task length, VM utilization, and delay sensitivity into linguistic values like "low," "medium," and "high.", a product inference engine - which uses some simple predefined fuzzy rules like, "If WAN bandwidth is low and task length is high, then execute on edge." to make offloading decisions, and

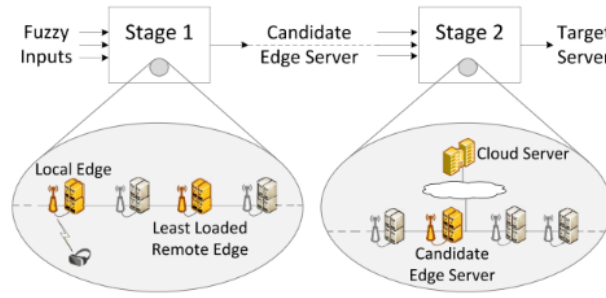


Figure 4: Two-stage fuzzy workload orchestration [4]

a centroid defuzzifier- which converts fuzzy decisions into precise outputs. This structured approach allows the system to achieve greater efficiency, adaptability, and overall performance improvements.

The simulation results showed that the fuzzy logic-based workload orchestrator outperformed other approaches in terms of the average number of failed tasks, the average service time, the average VM utilization, and the average network. It achieved fewer task failures by considering both computational capacity and network conditions, allowing it to make efficient task allocation decisions. The orchestrator provided the shortest service time under lighter loads but prioritized reducing task failures under higher loads, sometimes leading to longer processing delays. Also, Fuzzy orchestrator showed better resource utilization by effectively distributing workloads across edge servers, preventing CPU hotspots, and ensuring a balanced load by considering MAN delays. The analysis of task failure reveals that the fuzzy-based orchestrator balances failures more evenly across different causes, while other algorithms suffer disproportionate failures due to either MAN congestion or WAN delays.[4].

The fuzzy logic-based approach offers several strengths for edge workload orchestration. It efficiently handles dynamic, unpredictable environments and works well in real time due to its low computational complexity. Its rule-based structure offers a user-friendly interface, making it simple to implement and modify as needed. Additionally, the approach demonstrated strong performance across a range of tasks, including both delay-sensitive and compute-intensive tasks, providing flexibility and efficiency in diverse environments. Despite its strengths, the fuzzy logic-based orchestrator has some limitations. It assumes tasks are stateless, which may not be suitable for interdependent tasks. The fuzzy rules and membership functions were determined empirically, which could hinder scalability in some scenarios. Additionally, the impact of task length distributions on execution time is not fully addressed, and the task migration between edge and cloud servers has not been explored.[4].

The paper presents a fuzzy logic-based workload orchestrator designed to handle uncertainty and nonlinear environments with low computational complexity. It offers an efficient solution for task offloading and resource optimization. The study also acknowledges the need for further improvements, such as implementing intelligent optimization techniques to enhance the fuzzy membership functions and rules and exploring reinforcement learning for improved decision-making in extreme conditions

2.5 A novel approach for IoT tasks offloading in edge-cloud environments

The rapid growth of IoT devices and the vast amounts of data they generate have created significant challenges in computing and storage. While traditional cloud computing offers scalability, its centralized structure leads to high latency. This research aims to improve task offloading in Edge-Cloud environments by utilizing fuzzy logic algorithms for optimized scheduling. The proposed approach considers infrastructure and application characteristics like CPU demand, network load, delay sensitivity, resource utilization, and heterogeneity, dynamically adjusting to system changes to enhance workload distribution, reduce service time, and improve resource utilization for IoT applications in resource-limited settings. The proposed architecture consists of three interconnected layers: IoT de-

vices, edge computing nodes, and the cloud. IoT devices act as end users and generate tasks that need to be processed. These tasks are sent to the edge nodes, which can collaborate with other nodes or the cloud to process them. The system dynamically manages the offloading decisions through an Edge Controller, which is a layer between edge nodes and the cloud.

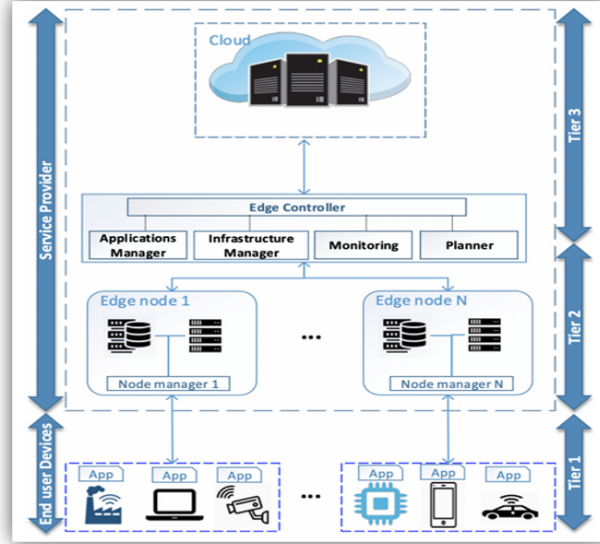


Figure 5: Edge-Cloud System [5]

The figure 5 illustrates a three-tier architecture. The Edge Controller(EC) is the core of the system that manages the task allocation process. The EC consists of different components: the Application Manager, Infrastructure Manager, Monitoring, and Planner. The Application Manager is responsible for handling the specific task requirements, such as its computational demand and latency constraints. The Infrastructure Manager manages the resources available on the edge nodes, including virtual machine(VMs), while the Monitoring component tracks the application tasks and computational resources during the execution. The Planner, based on real-time data, formulates a scheduling policy to efficiently allocate tasks to the most appropriate resources. It proposes and implements task-scheduling strategies. The Edge Controller's primary function is to collect data on the connected IoT devices, task specifications, and resource status at the edge and cloud levels. Then, using fuzzy logic algorithms, it makes real-time decisions regarding where tasks should be processed, edge nodes or cloud. [5].

The proposed solution utilizes an Edge Controller to collect real-time data from edge computing nodes, including the number of connected IoT devices, the task length, the computational cycles required, and the deadline constraints. The system continuously monitors the available resources at each edge node and employs fuzzy logic and task scheduling algorithms to determine the optimal offloading strategy. The fuzzy logic system analyzes four input variables: VM utilization, task length, network demand, and delay sensitivity. It first converts numerical values into linguistic terms (e.g., Low, Medium, High) through fuzzification. Then, a rule-based inference engine applies some "if-then" rules to determine the best task offloading strategy. Finally, the system defuzzifies the output, giving a decision whether to process the tasks locally at the edge, collaboratively across edge nodes, or in the cloud, ensuring an efficient task execution while minimizing latency.[5].

The proposed fuzzy logic-based approach was evaluated using simulation experiments and compared against existing scheduling algorithms, such as a utilization-based approach and algorithms by Sonmez and Flores. The results demonstrated that the proposed approach consistently main-

tained lower service time even as the number of IoT devices increased, whereas other methods suffered performance degradation under heavy loads. By considering resource heterogeneity along with task computation factors such as VM utilization, network demand, task length, and delay sensitivity, the approach dynamically allocates tasks to the most suitable resources, preventing overload and optimizing system performance. Furthermore, the proposed approach reduced network delays and task failures by preventing resource overload and network congestion. It also ensured a more balanced VM utilization, allowing effective management of computational resources. Unlike other methods, it maintained stable performance even under heavy system loads, avoiding significant performance degradation while enhancing the overall efficiency.[5].

The proposed approach has several strengths. It effectively adapts to the dynamic nature of the edge- cloud environment by considering both application-specific requirements and system parameters while also considering resource heterogeneity. This adaptability is essential for latency-sensitive applications that require real-time processing. The integration of fuzzy logic minimizes computational complexity, allowing rapid decision-making and improved system scalability. Moreover, the approach demonstrated strong performance in simulations, with reduced service time, reduced task failure, and efficient resource utilization.

However, the proposed approach has some limitations. The model assumes that task parameters like task length and data transfer requirements are known in advance, which may not always be the case in real-world scenarios. Additionally, the approach prioritizes reduced service time over efficient resource utilization, which could lead to underutilized resources at the edge and wastage of computation power. Future works could include reinforcement learning to improve decision accuracy and energy efficiency techniques, such as VM migration or auto-scaling, to enhance resource usage and meet application demands.[5].

Future work could extend the proposed approach by integrating complex computational resources, such as GPUs and FPGAs, to support applications with higher processing needs, such as augmented reality and video gaming. Additionally, task dependencies, represented as a Direct Acyclic Graph (DAG), can be integrated into the scheduling process to improve task offloading decisions.

The study's findings highlight a novel task offloading approach for latency-sensitive IoT applications in Edge-Cloud systems. This approach considers application characteristics and the dynamic, heterogeneous nature of resources to minimize service time and improve resource utilization. The proposed approach outperforms other scheduling algorithms by reducing service time and task failures and efficient resource usage, emphasizing the importance of considering application and system behavior for efficient task offloading. The study concludes that the proposed fuzzy logic approach is a viable solution for managing task offloading in edge-cloud systems, with room for further improvements to address energy efficiency and task prediction accuracy.

2.6 Adaptive Code Offloading for Mobile Cloud Applications: Exploiting Fuzzy Sets and Evidence-based Learning

With the increasing demand for computationally intensive mobile applications, code offloading has re-emerged as a key technique in mobile cloud computing(MCC). Recent research has focused on offloading solutions that bring cloud resources closer to mobile devices, addressing fundamental questions of what, when, and how to offload. Typically, these approaches involve cloud-hosted virtual machines while mobile devices manage partitioning strategies, decision logic, and implementation mechanisms for synchronization. However, existing models often overlook essential cloud capabilities such as elasticity, parallelization, and dynamic workload distribution.

To address these limitations, the paper proposes an advanced offloading framework that integrates cloud performance metrics into offloading decisions, ensuring optimal resource allocation and efficiency. Additionally, it introduces a fuzzy decision engine that adapts to changing network conditions, improving offloading decisions over time through evidence-based learning. Overall, this research aims

to bridge the gap between mobile and cloud environments, making MCC more scalable, responsive, and optimized for real-world applications.

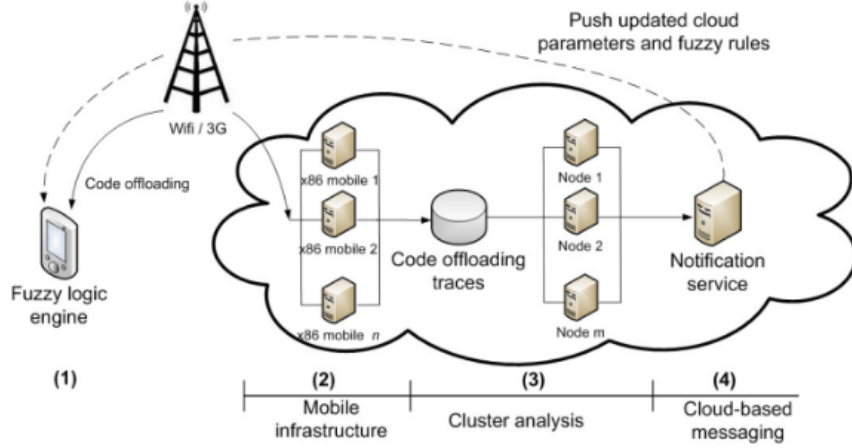


Figure 6: Evidence-based code offloading architecture)[6]

The system model for evidence-based mobile code offloading consists of four primary components as shown in figure 6. Initially, the Mobile Offloading Logic operates on the mobile device that uses a fuzzy logic-based decision engine that evaluates multiple parameters, like bandwidth, to determine the best offloading decision. Unlike binary approaches, fuzzy logic provides offloading decisions by assigning degrees of truth. Second, the Mobile Virtualization Infrastructure includes cloud-hosted virtualized instances that execute offloaded mobile components at the bytecode level. These instances also collect and store offloading traces, including device information, execution metrics, and network conditions. Additionally, they scale dynamically based on mobile traffic. Third, the Evidence-Based Control System consists of a cloud-based cluster that analyzes offloading traces using machine learning techniques such as clustering. To identify patterns that refine the fuzzy logic rules and improve decision-making over time. Finally, the Cloud-Based Messaging Framework enables asynchronous updates of offloading parameters. Real-time updates are pushed to mobile devices, by using cloud notification services, ensuring that the fuzzy logic engine remains updated with the changes in cloud performance and availability. All these components work together to provide a dynamic and adaptable architecture that incorporates both the cloud parameters as well as local device constraints to improve the effectiveness of mobile cloud computing.[6].

The paper presents a use case involving the delegation of resource-intensive mobile tasks to a hybrid cloud. It presents the Mobile Cloud Middleware(MCM) framework, to address interoperability, transparent delegation, asynchronous execution, and dynamic cloud resource allocation. The framework has been successfully used in a number of mobile cloud applications, including Bakabs, CroudSTag, and Zompopo. These applications use cloud capabilities to enhance user experience by optimizing processes like context-aware services, image recognition, and cloud resource management. Additionally, the approach uses a REST-based methodology to offload a task, in which execution information from past offloading cases further improves decision-making for future tasks.

The strengths of this paper lie in its consideration of the full potential of cloud computing in mobile cloud environments. It argues that cloud features such as elasticity, parallelization, and fine-grained billing should be better used in offloading models. The study presents a learning-based offloading approach that uses past offloading traces to refine mobile cloud decisions. While considering dynamic mobile cloud parameters, the approach aims to address challenges like device diversity, adaptive exe-

cution, and inconsistent profiling. Despite the strengths proposed system also has some limitations that must be addressed. Firstly, the system uses Google Cloud Messaging (GCM) for notification services. Which may not be acceptable for real-time applications as it has an average delivery time of approximately 10 seconds. Second, the evidence-based learning mechanism is still underdeveloped as the parallel version of the neuronal network algorithm for analyzing code offloading traces is in progress. Additionally, the paper primarily focuses on mobile and cloud, but relying solely on cloud-based task offloading can lead to latency issues, which is a significant challenge in real-time applications.

The future work will focus on fully implementing the existing simulated component of the mobile cloud architecture and demonstrating it through case studies. This includes developing a parallel neural network approach for big code offloading traces analysis on a MapReduce cluster, along with exploration of some other machine learning techniques. Also, the proposed fuzzy logic-based mechanism will be compared with other algorithms to evaluate the energy-saving benefits of separating decision processes between mobile and cloud.

To summarize, this paper explores the use of fuzzy logic to improve code offloading decisions in mobile cloud environments. It proposes a mobile cloud architecture that enhances device performance and energy efficiency by considering the cloud's flexibility, parallelization, and other capabilities. The paper focuses on using offloading traces as knowledge to allow mobile devices to make smarter offloading decisions in diverse conditions.

Below is a table summarizing task offloading approaches, which is my own contribution to the thesis. It highlights the proposed algorithm, decision-making entities, task offloading parameters categorized into static and dynamic types, compared schemes, and evaluation metrics. This table provides a concise overview of existing methods and their respective features, facilitating a better understanding of task-offloading strategies in edge-cloud environments.

The table includes:

1. **Decision-Making Entities:** Identifies the components responsible for making task offloading decisions (e.g., Edge Devices, Edge Controllers).
2. **Static Task Offloading Parameters:** Fixed parameters such as task length or type of application.
3. **Dynamic Task Offloading Parameters:** Varying parameters like CPU availability, bandwidth, or VM utilization.
4. **Compared Schemes:** Highlights alternative approaches used for comparison.
5. **Evaluation Metrics:** Defines the criteria for assessing performance, such as processing time, energy consumption, and delay.

Table 1: Summary of Task Offloading Approaches

Paper	Proposed Algorithm	Task Offloading Decisions	Static Task Offloading Parameters	Dynamic Task Offloading Parameters	Compared Schemes	Evaluation Metrics
A novel approach for IoT tasks offloading in edge-cloud environments	Fuzzy logic	Edge Controller	Task length, Network demand, Delay sensitivity	VM utilization	Utilization-Based, Sonmez, Flore	Service time, Processing delay, Network delay, Task failure and Average VM utilization
Fuzzy Workload Orchestration for Edge Computing	Fuzzy Logic	Edge Orchestrator	Task length, Network demand, Delay sensitivity	VM utilization	Modified Flores-based, Utilization-based, Bandwidth-based, Hybrid	Average Service Time, Average number of Failed tasks, Average VM utilization, Average Network delay
Adaptive Code Offloading for Mobile Cloud Applications	Fuzzy logic	Mobile Device	-	-	-	-

Utilization-Based strategy evaluates the CPU usage of edge and cloud resources. If CPU utilization is below 80%, the task is offloaded to the edge; if it's higher, it is sent to the cloud. The Hybrid approach combines both network-based and utilization-based task offloading algorithms, that is if both the WAN bandwidth exceeds 6 Mbps and the average VM utilization exceeds 80%, offload the task to the cloud; otherwise, offload to the edge server. These thresholds of 6 Mbps and 80% are empirical values determined by the developers of EdgeCloudSim.[4] In the EdgeCloudSim these three algorithms are combined in single algorithm.

Algorithm: *Network based, Utilization based and Hybrid*

Input: offload policy P , incoming task T_i , WAN bandwidth ($wanBW$), Edge Utilization ($utilization$)

Output: Select the Target Edge Server (R_{edge}) or Cloud server (R_{cloud})

```

1.  If  $P = \text{Network Based}$  then
2.      If  $wanBW > 6$  then
3.           $R = R_{cloud}$ 
4.      Else
5.           $R = R_{edge}$ 
6.
7.  Else If  $P = \text{Utilization Based}$  then
8.      If  $utilization > 80$  then
9.           $R = R_{cloud}$ 
10.     Else
11.          $R = R_{edge}$ 
12.
13.  Else If  $P = \text{Hybrid}$  then
14.     If  $wanBW > 6$  and  $utilization > 80$  then
15.          $R = R_{cloud}$ 
16.     Else
17.          $R = R_{edge}$ 

```

Figure 8: psuedocode: Network-based, Utilization based, Hybrid

3.1.3 Fuzzy

Fuzzy logic is used here to decide whether a task should be offloaded to an edge server or the cloud based on various parameters. It works in two stages, in the first stage, it selects the best edge server based on parameters like network delay, edge utilization, and remote edge utilization, using 27 different predefined fuzzy rules. In the second stage, it evaluates task-specific parameters such as task size, delay sensitivity, bandwidth, and VM utilization. With 81 additional fuzzy rules, it calculates membership values for both the cloud and edge servers. If the membership value is lower than 50, the task is offloaded to the edge; otherwise, it is offloaded to the cloud.[4]

Algorithm 1: Fuzzy Logic for Offloading to the Best Target Edge or Cloud Server

INPUT: task T_i with parameters $T_{manDelay}$, $T_{edgeutil}$, and $T_{nearestEdgeU}$, Also T_{size} , T_{wanbw} , $T_{avgEdgeUtil}$ and $T_{delay-sensitivity}$

OUTPUT: Select the Target Edge Server($R_{nearest}$, R_{remote}) or Cloud server (R_{cloud})

```

1: for Task  $T_i$  do
2:    $F = \text{FuzzyLogicSystem}(T_{manDelay}, T_{edgeutil}, \text{and } T_{nearestEdgeU})$ 
3:   if  $F > 50$  then
4:     Allocate  $T_i$  on  $R_{edge} = R_{remote}$ 
5:   else
6:     Allocate  $T_i$  on  $R_{edge} = R_{nearest}$ 
7:   end if
8: end for
9: for Task  $T_i$  do
10:   $F = \text{FuzzyLogicSystem}(T_{size}, T_{wanbw}, T_{avgEdgeUtil} \text{ and } T_{delay-sensitivity})$ 
11:  if  $F > 50$  then
12:    Allocate  $T_i$  to  $R_{cloud}$ 
13:  else
14:    Allocate  $T_i$  to  $R_{edge}$ 
15:  end if
16: end for

```

Figure 9: psuedocode: Fuzzy Algorithm

4 Analysis of the Task offloading approaches

This section gives an in-depth analysis of several existing task-offloading algorithms based on different performance metrics, which are obtained by running several iterations. The results for the analysis are generated using the EdgeCloudSim framework [4]. For the Simulation Scenarios, six edge servers are used and the number of IoT devices are varied.

4.0.1 Number of Failed tasks

The average task failure percentage is one of the most crucial performance criteria for task offloading. In simulations, the tasks fail due to four reasons which are mobility, lack of VM capacity on the edge servers, and lack of MAN and WAN capacity. The failures due to mobility are common for all approaches since if the users change their location before getting the response of the previously requested tasks, related tasks fail because the users are not connected to the related WLAN. The average failed task percentage for 1000 mobile devices is given in 10. Performance of all the approaches except the Next-Fit approach is similar when the system is lightly loaded. However, the fuzzy-based approach gives the best results compared to all the other algorithms as the load increases. The fuzzy-based balances both computational and networking resources and dynamically adjusts its offloading decision based on the condition of those resources. The Next-Fit gives the worst results as it focuses on task allocation without considering network or resource congestion at the server. As a result, the system becomes overwhelmed with increasing load, leading to congestion at the edge level and, thus more failed tasks. The Utilization Based gives the second worst results because it uses a threshold i.e., 80 % and after the computational resources of the edge are congested, incoming tasks are sent to the cloud. As a result, after some time most of the tasks drop on the network due to the WAN congestion. While, the Network-Based approach shows a low task failure percentage, where most of

the tasks drop due to the MAN capacity because it only considers the WAN bandwidth and starts offloading tasks to the edge server after the WAN is congested.

Failed Task

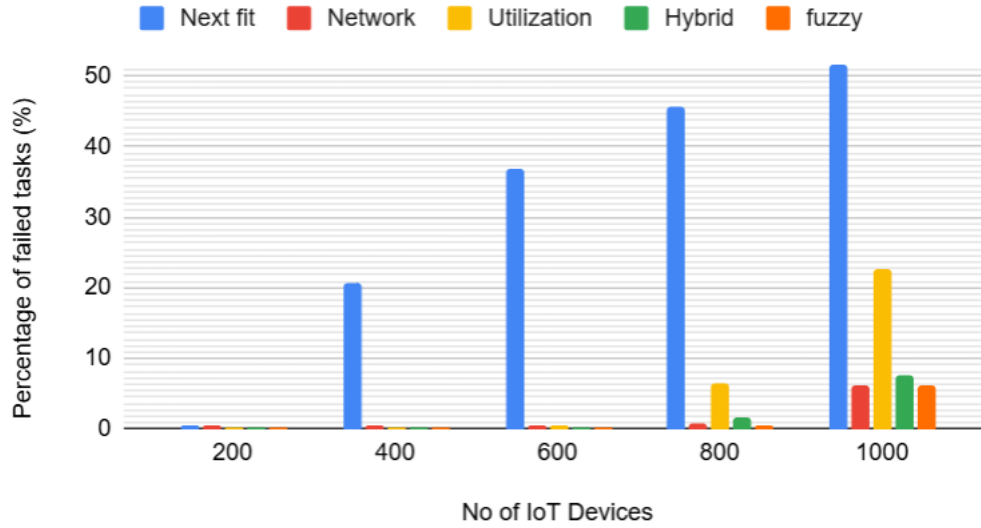


Figure 10: Number of Failed Tasks

4.0.2 Network Delay

Network Delay

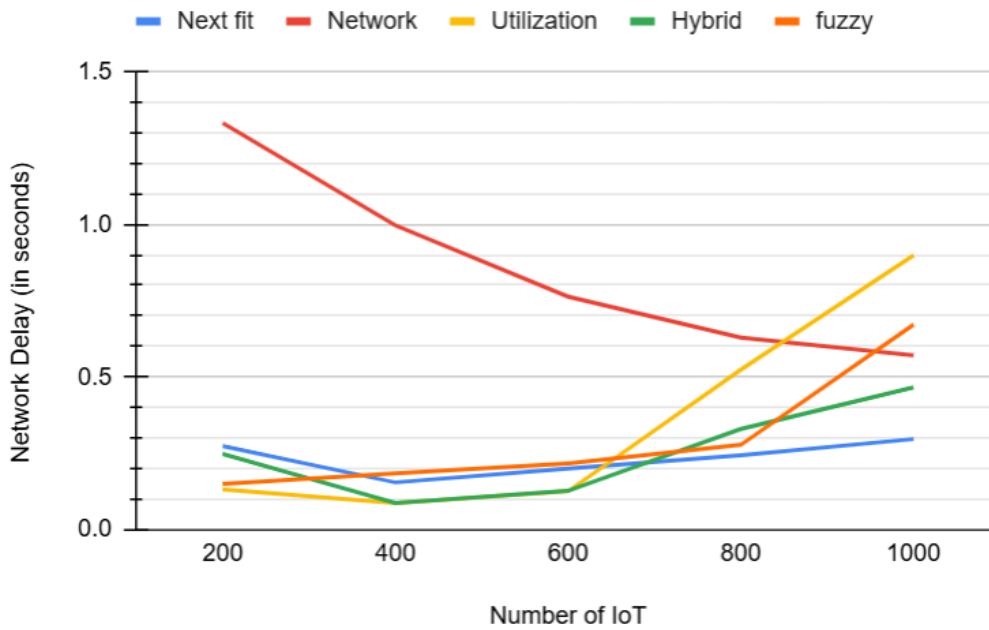


Figure 11: Network Delay

Network delay refers to the time required for a task to travel to and return from the cloud. It is a crucial performance metric as it directly impacts the overall task completion time. The network delay for various algorithms is illustrated in Figure 11.

Initially, when the system load is low, network-based algorithms exhibit the worst performance. This occurs because the algorithm assumes that WAN bandwidth is inversely proportional to WAN delay. When the load is light, the WAN delay is minimal, leading to an increase in available bandwidth. Since this increased WAN bandwidth exceeds the threshold of 6 Mbps, more tasks are offloaded to the cloud. Consequently, this results in higher network delay due to the increased offloading of tasks to the cloud. Whereas other algorithms show similar results at lower loads. But at higher loads, utilization-based shows the worst results because when the CPU resources at the Edge servers become congested, it offloads tasks to the cloud, increasing the network delay. The hybrid approach uses parameters of both network and utilization-based algorithms and shows better results than them. Even though next-fit shows the least network delay, it cannot be considered the best performing considering the amount of tasks it drops at a higher load, which is almost 50%. While Fuzzy gives a balanced performance with a gradual increase in the network delay, it can be considered the best performer.

4.0.3 Processing Delay

Processing delay refers to the time required to execute a task and directly impacts the overall task completion time. The processing delay for various algorithms is illustrated in Figure 12.

The next fit is the worst performer. Although its processing time initially decreases, it becomes inefficient when the number of tasks reaches 400. At this point, the edge server utilization reaches about 80%, which means the servers are overburdened. As a result, a significant number of tasks begin to fail due to congested CPU resources, reducing the total number of processed tasks and, consequently, lowering the processing time, though this decrease does not reflect improved performance.

For lower loads, the network-based approach gives the best result because it prioritizes offloading the tasks to the cloud, which has more processing power, and this leads to shorter processing times. While fuzzy, utilization-based and hybrid primarily offload tasks to the edge servers, which have limited computing power compared to the cloud. As the number of IoT devices increases, the workload on the edge servers increases due to a larger number of tasks. This increased load slows down processing, resulting in longer processing times.

4.0.4 Service time

Service Time is the total completion time of a task, it includes both network delay and processing delay. Figure 13 illustrates the service time across various algorithms.

When the system load is light, the Fuzzy-based approach delivers the best performance, as it considers both computational and network resources to make more informed decisions. In contrast, the Next Fit approach performs the worst. It continues offloading tasks to edge servers until they reach saturation, raising the processing time of tasks and increasing task failures. As the number of failed tasks rises, the overall service time initially increases but then declines beyond a certain number of IoT devices due to fewer successful task completions.

The Utilization-based and hybrid algorithms offload tasks to the cloud when the CPU resources of the edge servers become congested. However, since the WAN delay is higher than the processing delay, the overall service time increases, resulting in poor performance. The network-based approach prioritizes offloading tasks to the cloud. As network congestion increases, network delay rises, due to which the algorithm shifts task offloading to the edge server, thereby improving performance. While the fuzzy logic-based algorithm yields service times similar to the utilization-based and hybrid approaches, it has

Processing Time

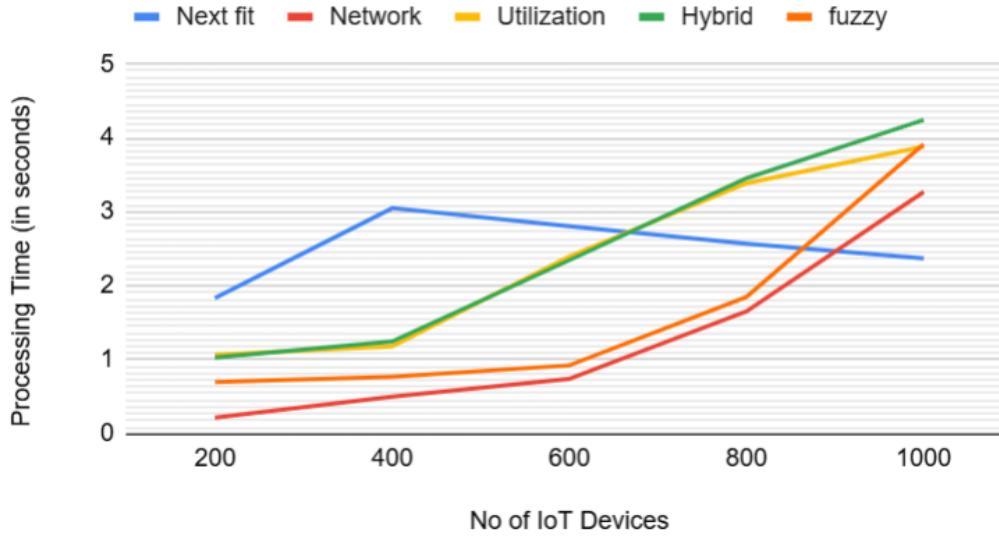


Figure 12: Processing Delay

Service Time

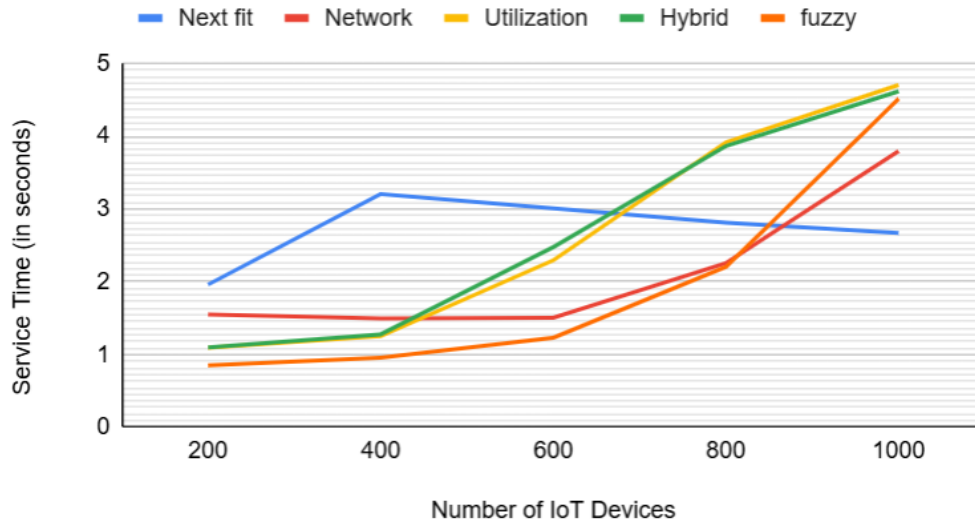


Figure 13: Service Time

a lower average task failure rate. This suggests that, under system overload conditions, it maintains lower task failure, causing it to have a longer service time than other algorithms at higher loads.

4.0.5 Edge Server Utilization

Edge utilization is defined as the average utilization of virtual machines running on the edge servers and is an important parameter for analysis. The edge server utilization for various algorithms is shown in Figure 14.

The next fit approach exhibits the highest edge server utilization, as it offloads a task to an edge

server if it has enough capacity without considering any resources, which leads to excessive edge utilization and eventual resource exhaustion.

In contrast, the network-based approach demonstrates lower edge utilization under lighter loads. When the number of IoT devices is low, network delay remains minimal, WAN bandwidth is higher, and tasks are offloaded to the cloud, provided the WAN bandwidth exceeds 6 Mbps. Fuzzy logic-based algorithm shows similar results as it takes various network and computational parameters into account for decision making.

Meanwhile, the utilization-based and hybrid algorithms prioritize offloading to the edge servers since they consider a threshold of 80 percent edge utilization before sending a task to the cloud.

Edge Server Utilization

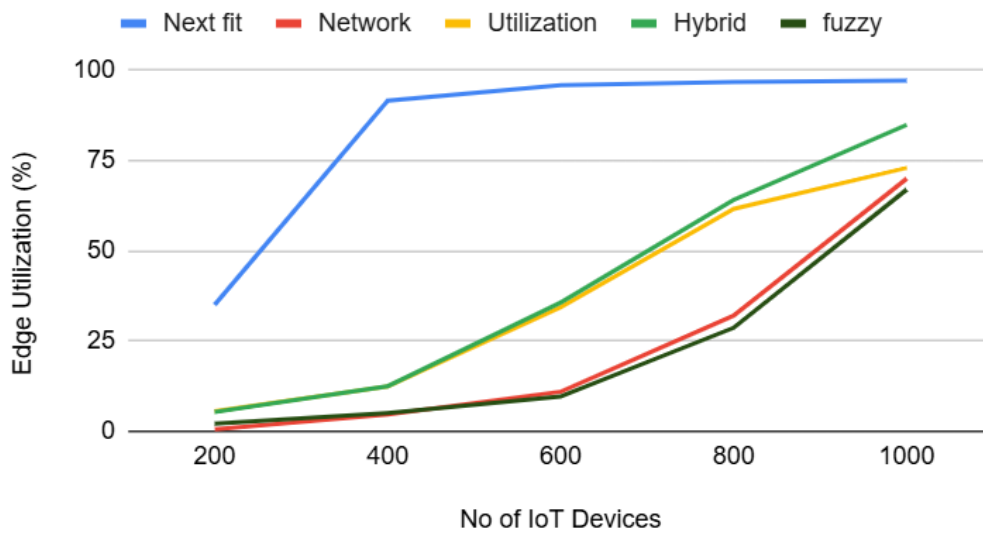


Figure 14: Edge Server Utilization

4.0.6 Cloud Server Utilization

Cloud utilization refers to the percentage of the cloud resources that are being actively used. When the loads are lighter, the network-based algorithm exhibits the highest cloud utilization. This is because, as previously discussed, lower WAN delay leads to greater available WAN bandwidth, allowing more tasks to be offloaded to the cloud. This utilization continues to rise until the number of IoT devices reaches 600, then declines because with the increase in the number of IoT devices, the WAN network gets congested, resulting in increased network delay.

The utilization-based and hybrid approaches primarily offload tasks to edge servers. However, when the average utilization exceeds 80%, under higher loads, it starts offloading the task to the cloud. While next-fit and fuzzy-based approaches show an almost exponential growth in cloud utilization. Among them, the fuzzy-based approach achieves the highest cloud utilization, indicating a more balanced utilization of resources across cloud and edge servers.

Cloud Server Utilization

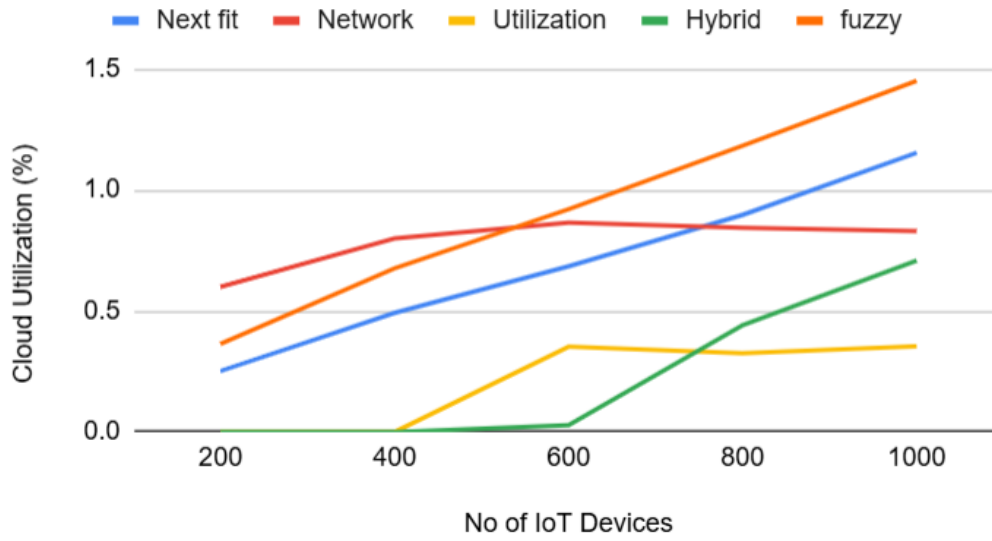


Figure 15: Cloud Server Utilization

Hence, Fuzzy-based shows better results for all the different metrics by effectively utilizing the resources considering both computational and network resources in the decision making.

5 Conclusion

The project provides a comprehensive analysis of several existing task offloading schemes in edge-cloud computing systems, identifying key methodologies, strengths, and limitations.

References

- [1] H. El-Sayed, S. Sankar, M. Prasad, D. Puthal, A. Gupta, M. Mohanty, and C.-T. Lin, “Edge of things: The big picture on the integration of edge, iot and the cloud in a distributed computing environment,” *IEEE Access*, vol. 6, pp. 1706–1717, 2018.
- [2] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue, “All one needs to know about fog computing and related edge computing paradigms: A complete survey,” *Journal of Systems Architecture*, vol. 98, p. 289–330, Sep. 2019. [Online]. Available: <http://dx.doi.org/10.1016/j.sysarc.2019.02.009>
- [3] K. Cao, S. Hu, Y. Shi, A. W. Colombo, S. Karnouskos, and X. Li, “A survey on edge and edge-cloud computing assisted cyber-physical systems,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 11, pp. 7806–7819, 2021.
- [4] C. Sonmez, A. Ozgovde, and C. Ersoy, “Fuzzy workload orchestration for edge computing,” *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 769–782, 2019.
- [5] J. Almutairi and M. Aldossary, “A novel approach for iot tasks offloading in edge-cloud environments,” *Journal of cloud computing*, vol. 10, no. 1, p. 28, 2021.
- [6] H. Flores and S. Srirama, “Adaptive code offloading for mobile cloud applications: exploiting fuzzy sets and evidence-based learning,” in *Proceeding of the fourth ACM workshop on Mobile cloud computing and services*, ser. MobiSys’13. ACM, Jun. 2013. [Online]. Available: <http://dx.doi.org/10.1145/2482981.2482984>